

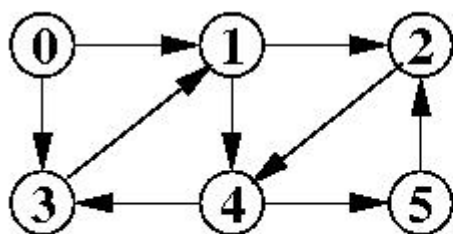
Grafos: Representación mediante Matriz de Adyacencia

Definición / Idea Básica

Sea m la cantidad de nodos del grafo. (Es preciso conocer m de antemano).

Se define como matriz de adyacencia del grafo G , a la matriz cuadrada booleana de $m \times m$, donde m es la cantidad de puntos del conjunto P

Representación:



O\D	0	1	2	3	4	5
0	0	1	0	1	0	0
1	0	0	1	0	1	0
2	0	0	0	0	1	0
3	0	1	0	0	0	0
4	0	0	0	1	0	1
5	0	0	1	0	0	0

En un grafo dirigido (dígrafo) las filas representan los orígenes y las columnas los destinos.

Representación Formal:

$$\text{Ag: } \forall_{i,j} \in P: \begin{aligned} a_{ij} &= 1, (i,j) \in R \\ a_{ij} &= 0, (i,j) \notin R \end{aligned}$$

Definición de la Estructura En C++ - Alternativa Básica

```
#define M 6  
  
int A[M][M];  
char etiqueta[M];
```

Definición de la Estructura En C++ - Alternativa Completa

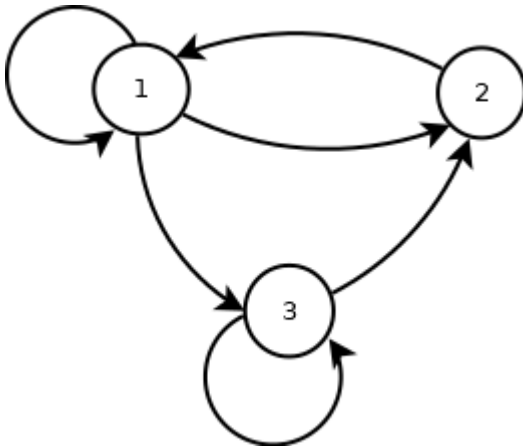
```
#define M 6  
  
int A[M][M];  
struct InfNodo datosNodo[M];  
struct infArco datosArco[M][M];
```

Características

- M se debe conocer de antemano, es decir se usan para grafos poco volátiles.
- Permiten ejecutar consultas rápidas.
- Estas matrices no se almacenan, se alojan en memoria.

Potencia n-ésima de la matriz de adyacencia de un grafo

Significado de la matriz A^n



Calculemos, por ejemplo, A^3

$$A^2 = A \cdot A$$

$$A^3 = A^2 \cdot A$$

Con SCILab

```
-->A=[1 1 1; 1 0 0; 0 1 1]
```

A =

1.	1.	1.
1.	0.	0.
0.	1.	1.

```
-->A^3
```

A^3 =

4.	4.	4.
2.	2.	2.
2.	2.	2.

α_{ij} : Es el elemento correspondiente a fila i y columna j de A^3 .

Entonces, α_{ij} : representa el número de caminos de **n tramos** que salen de i y llegan a j .

A^3 =

4.	4.	4.
2.	2.	2.
2.	2.	2.

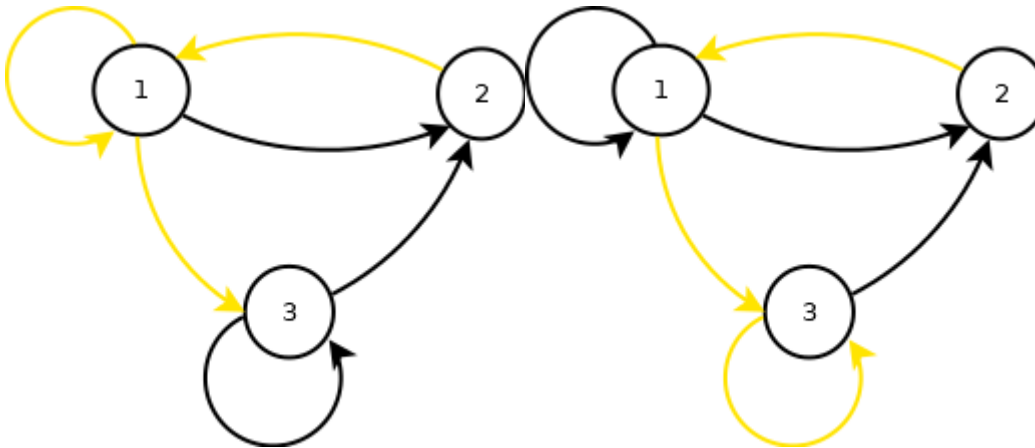
2.

α_{31}

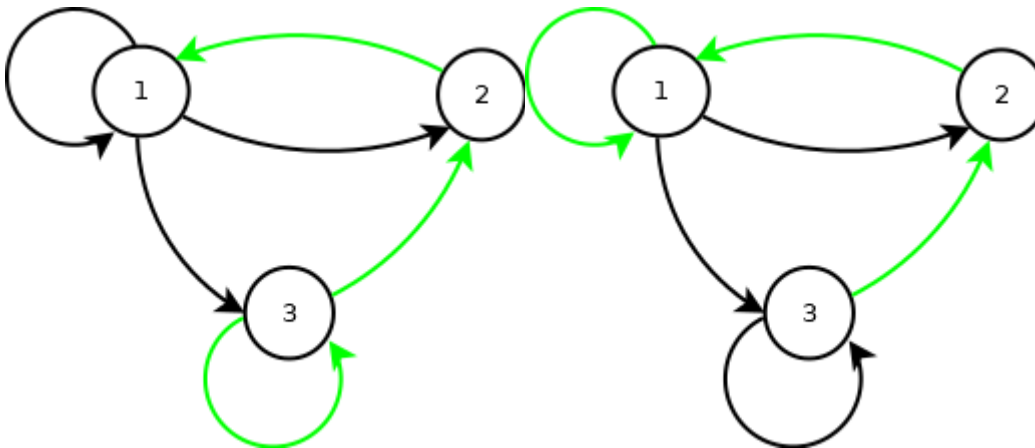
Los dos caminos de $i=2$ a $j=3$ son $\{2,1,1,3\}$ $\{2,1,3,3\}$

Los dos caminos de $i=3$ a $j=1$ son $\{3,3,2,1\}$ $\{3,2,1,1\}$

α_{23} Los dos caminos de $i=2$ a $j=3$ son $\{2,1,1,3\}$ $\{2,1,3,3\}$



α_{31} Los dos caminos de $i=3$ a $j=1$ son $\{3,3,2,1\}$ $\{3,2,1,1\}$



Ejemplo

```
int A[6][6];

//inicializar todo a false
for(int i=0;i<6;i++)
    for(int j=0;j<6;j++)
        A[i][j] = 0;

//true si hay enlace origen -> destino
A[0][1] = 1;
A[1][2] = 1;
A[1][4] = 1;
A[2][4] = 1;
A[3][1] = 1;
A[4][3] = 1;
A[4][5] = 1;
A[5][2] = 1;
```